

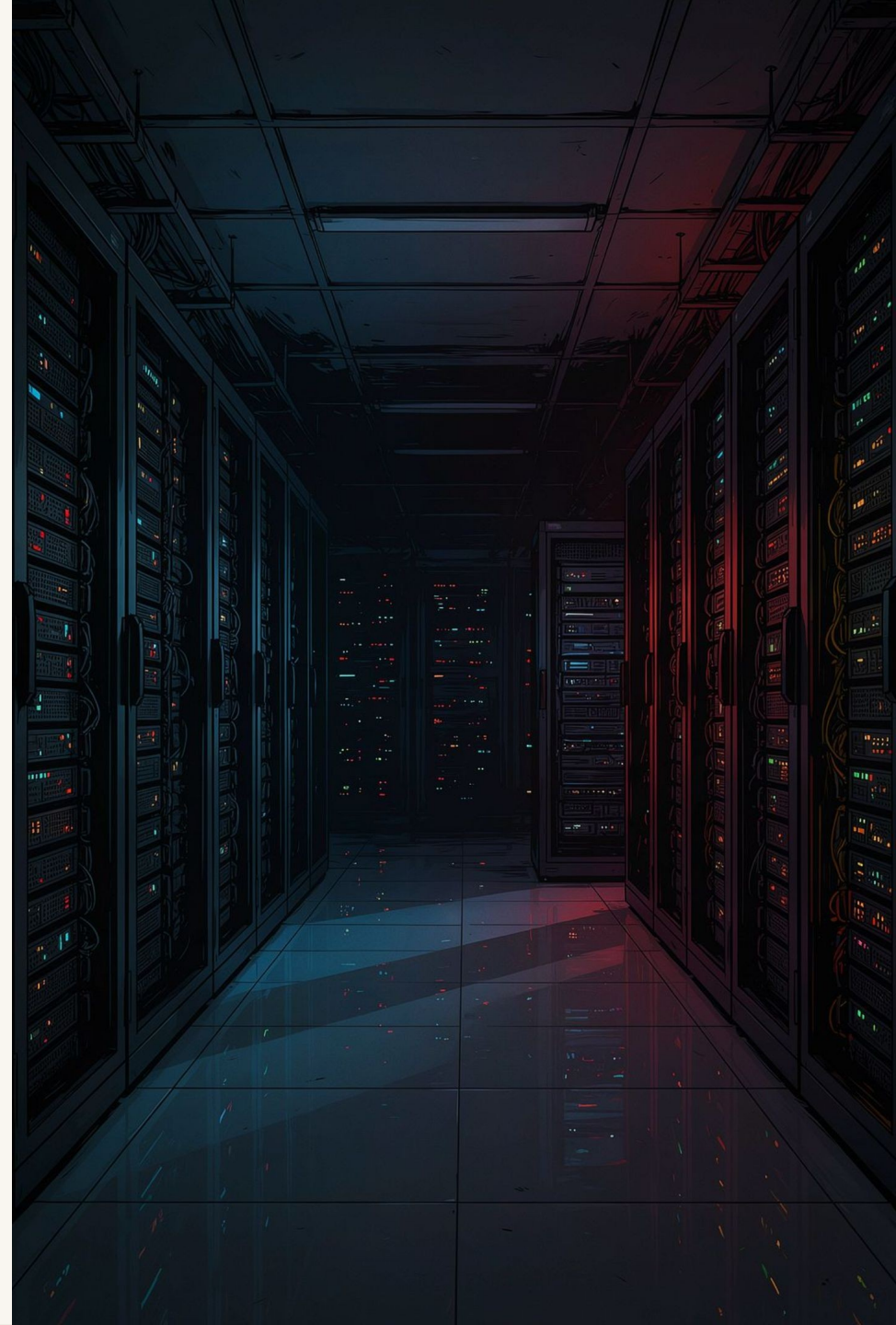
Jenkins Administration

Managing and Maintaining Enterprise Jenkins Servers

A comprehensive training guide covering tool configuration, plugin management, maintenance strategies, and enterprise best practices for production Jenkins environments.

DEVOPS TRAINING

ENTERPRISE EDITION



Tool Configuration Overview

Access global tool settings via **Manage Jenkins → Tools**. Configure all build tools in one centralized location to ensure consistency across all agents and pipelines.



JDK

Java Development Kit versions for builds



Maven

Project build and dependency management



Gradle

Flexible build automation tool



Git

Source code version control



Docker

Container build and runtime



NodeJS

JavaScript runtime environment



Sonar Scanner

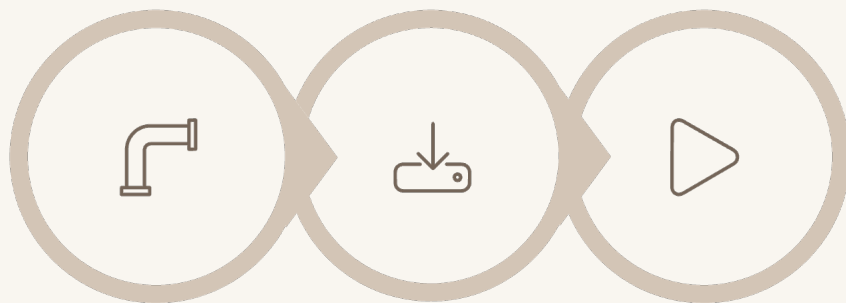
Static code analysis integration



Navigate to **Manage Jenkins → Tools** to configure globally. Changes apply to all pipelines immediately.

Tool Auto Installation

Jenkins can automatically download and install **JDK, Maven, Gradle, and NodeJS** on demand — eliminating manual setup on each agent.



Pipeline
Request

Jenkins
Download

Build Starts

Standard Versions

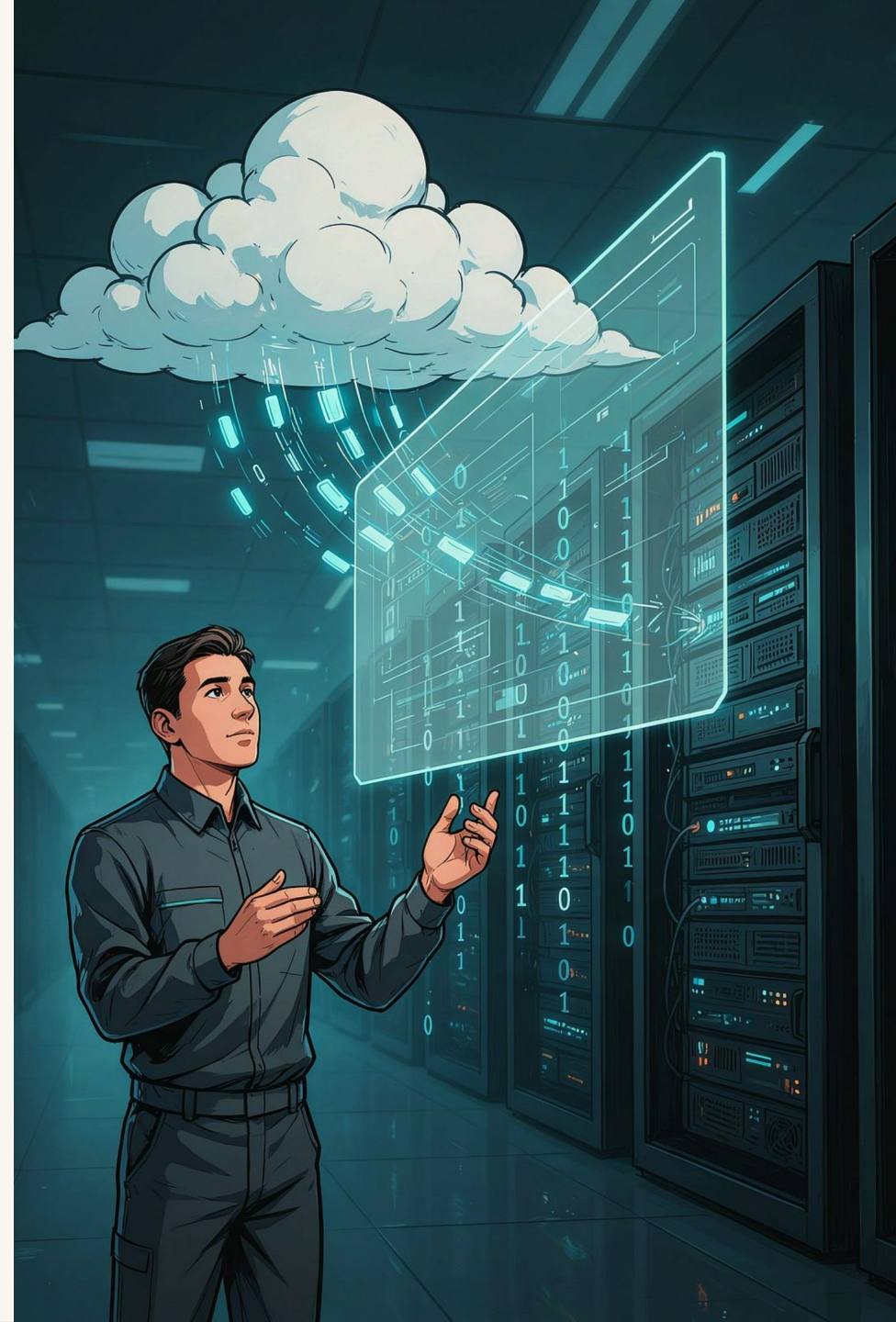
Enforce consistent tool versions across all environments

Faster Setup

No manual installation required on new agents

Consistency

Identical builds regardless of agent configuration



JDK Configuration

Set the JDK name and installation path in **Manage Jenkins → Tools → JDK**. Use a descriptive name like `JDK17` to reference it in pipelines.

Path: `/usr/lib/jvm/java-17`

📄 Use auto-install for new agents. Manual path is preferred for pre-configured enterprise images.

Pipeline Declaration

```
pipeline {
  agent any
  tools {
    jdk 'JDK17'
  }
  stages {
    stage('Build') {
      steps {
        sh 'java -version'
      }
    }
  }
}
```

The `tools` block ensures the correct JDK is available in the pipeline environment automatically.

Maven & Git Configuration

Maven Setup

Configure Maven under **Manage Jenkins → Tools → Maven**.
Choose between **Automatic** (Jenkins downloads) or **Manual** (specify local path).

Example name: Maven3.9

```
tools { maven 'Maven3.9' }
```

Git Setup

Specify the Git executable path. Verify installation before saving.
Requires the **Git plugin** to be installed.

Executable: /usr/bin/git

Verify: git --version

```
checkout scm
```

Enterprise Tip

Always use named tool configurations in pipelines. Avoid hardcoding paths — named tools allow centralized version upgrades without modifying individual Jenkinsfiles.



Plugin Management Overview

Plugins are the backbone of Jenkins functionality. **Without plugins, Jenkins is extremely limited** — they unlock integrations with every major DevOps tool.



Source Control

Git, GitHub, Bitbucket integrations



Containers

Docker, Kubernetes orchestration



Cloud & IaC

AWS, Terraform, Ansible support



Quality & Notify

SonarQube, Slack, Email Extension

Plugin Manager Interface

Navigate to **Manage Jenkins** → **Plugins** to access the full Plugin Manager. Four key sections organize all plugin operations.

1

Updates

View and apply available updates for installed plugins

2

Available

Browse and install new plugins from the Jenkins marketplace

3

Installed

Review all currently installed plugins and their versions

4

Advanced

Upload custom plugins and configure update site URLs



The **Advanced** tab is essential in air-gapped enterprise environments where direct internet access is restricted.

Essential Plugins for Enterprise Jenkins

Install these recommended plugins to enable a fully functional enterprise CI/CD environment from day one.

Pipeline & Source Control

- Pipeline
- Git & GitHub
- Blue Ocean
- Pipeline Utility Steps

Containers & Cloud

- Docker
- Docker Pipeline

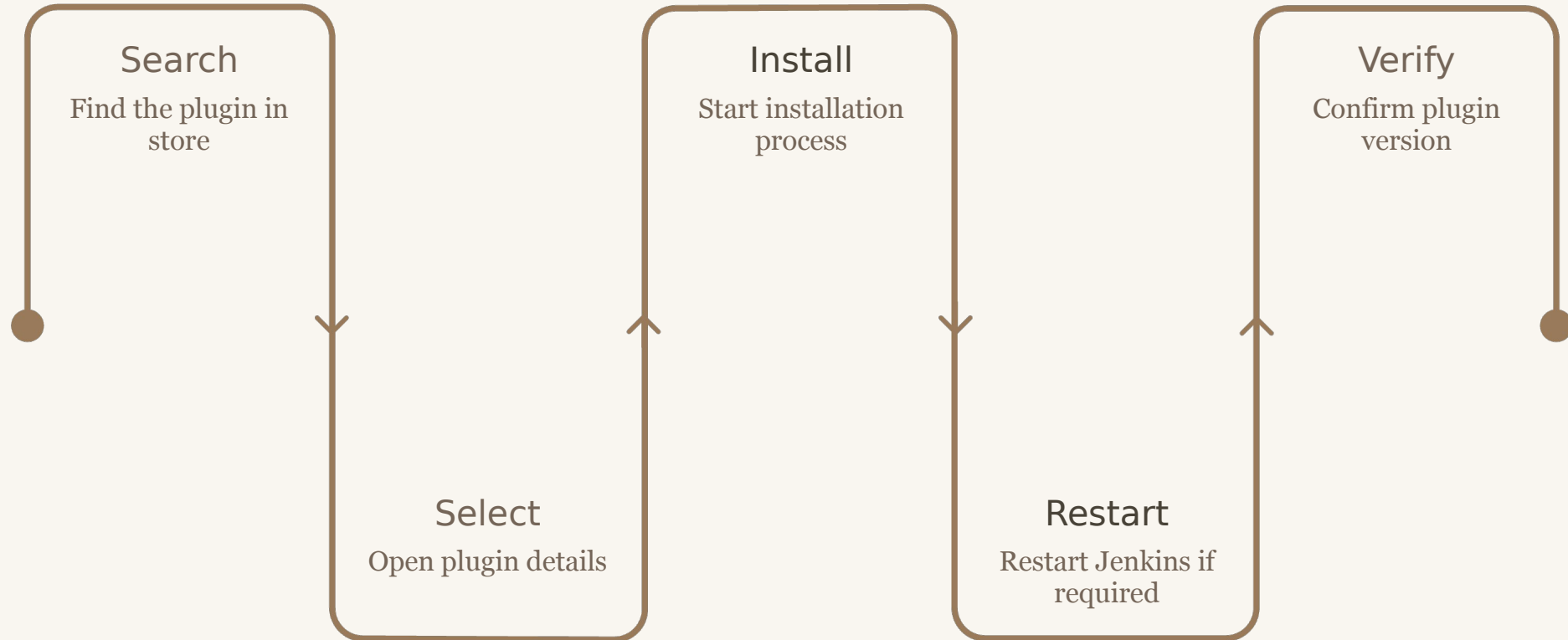
Security & Access

- Credentials Binding
- SSH Agent
- Role Strategy
- Configuration as Code

Productivity

- Workspace Cleanup
- AnsiColor
- Build Timeout
- Email Extension

Plugin Installation Process



Read Release Notes

Review changelogs for breaking changes before installing any plugin update

Verify Compatibility

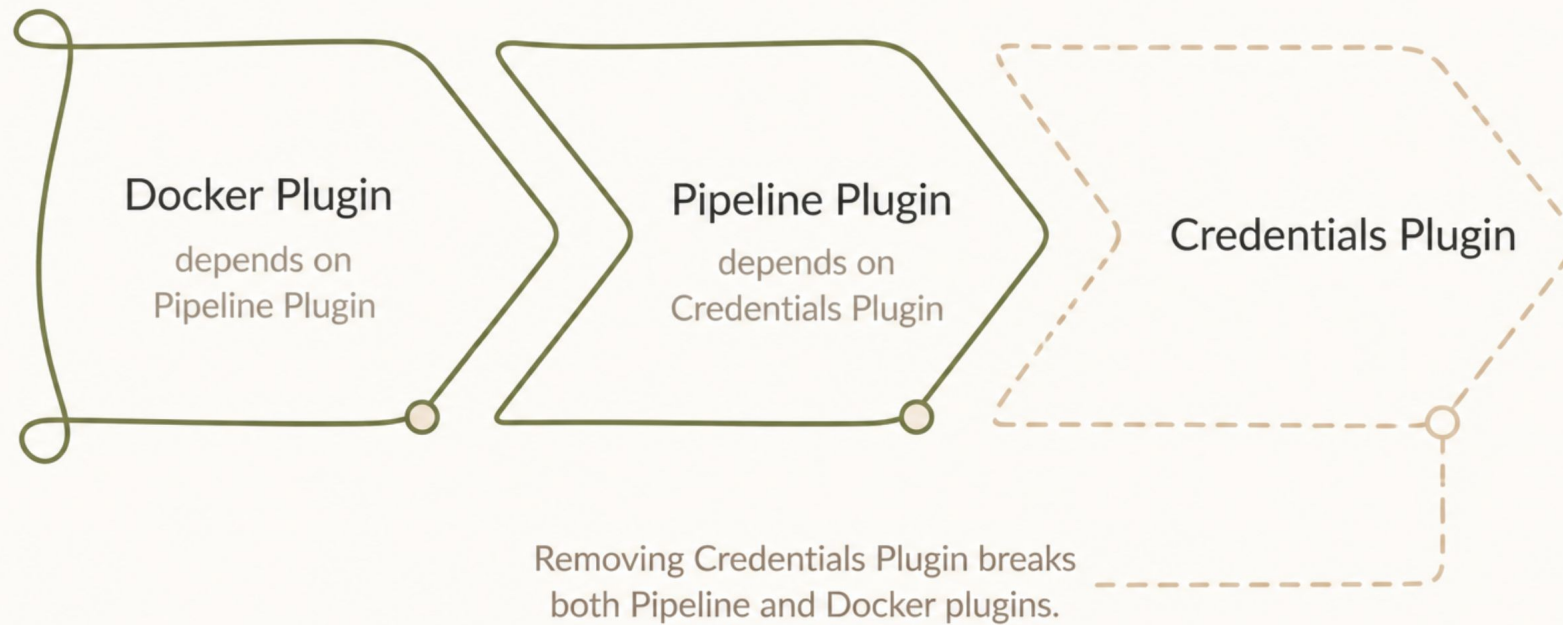
Confirm the plugin version supports your Jenkins core version

Test in Staging

Always validate new plugins in a non-production environment first

Plugin Dependencies

Jenkins plugins rarely operate in isolation. Most depend on other plugins — removing one can silently break several others.



⚠ Always check the **dependency tree** before removing or downgrading a plugin. Use the Installed tab to inspect what each plugin requires.

Plugin Best Practices

Do

- Update plugins regularly
- Remove unused plugins
- Test updates in staging
- Backup before upgrades
- Monitor security advisories

Avoid

- Installing unnecessary plugins
- Updating production during business hours
- Ignoring dependency warnings



Subscribe to the **Jenkins Security Advisory** mailing list to receive timely alerts about plugin vulnerabilities.

Jenkins Maintenance Schedule

A structured maintenance cadence keeps Jenkins stable, secure, and performant in enterprise environments.



Daily

- Monitor builds
- Check disk usage
- Review logs



Weekly

- Update plugins
- Remove old workspaces
- Backup jobs



Monthly

- Upgrade Jenkins core
- Review security
- Audit users
- Validate backups



Enterprise Best Practices



Security

- Matrix Authorization
- RBAC with Role Strategy
- HTTPS enforced
- Credentials Store



Performance

- Dedicated build agents
- Workspace cleanup
- SSD storage



Reliability

- Scheduled backups
- Monitoring & alerting
- HA architecture

Live Demonstration

Follow along with these hands-on demos to reinforce the configuration and administration concepts covered in this training.

01

Explore JENKINS_HOME

Navigate the Jenkins home directory structure and key configuration files

02

Configure Global Settings

Set system-level configurations in Manage Jenkins → System

03

Configure JDK & Maven

Add JDK17 and Maven3.9 tool configurations with auto-install

04

Install Git Plugin

Search, install, and verify the Git plugin from Plugin Manager

05

Restart & Verify

Safely restart Jenkins and confirm plugin functionality in a test pipeline

Key Takeaways

This training covered the essential pillars of enterprise Jenkins administration — from tool configuration to long-term maintenance.



Understand Jenkins Home

Know the directory structure and where critical configuration lives



Configure Global & System Settings

Centralize tool and system configuration for all agents and pipelines



Install and Manage Plugins

Use the Plugin Manager strategically — install, update, and audit regularly



Follow Enterprise Best Practices

Security, performance, and reliability are non-negotiable in production



Maintain a Stable Jenkins Environment

Daily, weekly, and monthly routines keep Jenkins healthy long-term

Jenkins Administration Checklist

Use this checklist to validate your Jenkins environment meets enterprise-grade administration standards before going to production.

- ✓ Backup configured
- ✓ HTTPS enabled
- ✓ Role-based access configured
- ✓ Credentials secured
- ✓ Build agents healthy
- ✓ Plugins updated
- ✓ Logs monitored
- ✓ Disk space monitored
- ✓ Build cleanup configured
- ✓ Backup tested

📄 Run this checklist after every major Jenkins upgrade or infrastructure change to ensure nothing is missed.

